

Competitive Programming

Section 39: Heaps and Priority Queues

Competitive Programming Course

2026-08-30

Table of contents

1	Priority queues as frontier managers	3
2	What a binary heap actually stores	4
2.1	Why heaps are stored in arrays	4
2.2	Push and pop	5
2.3	Repairing the heap after a push	5
2.4	Repairing the heap after a pop	6
3	Heap versus sorted container	7
4	Two heaps for medians	8
5	Sweep with ending times	9
5.1	Why the maximum heap size is the answer	9
6	K-way merging	10
6.1	Why the popped sum is globally next	11
7	Optimal merging: repeatedly combine the two smallest	12
8	Python and C++ heap details	13
9	Checklist	14
10	Local exercises	15
11	How to use the checker	16
11.1	Running median: preserve two halves	16
11.2	Course rooms: a heap is the active frontier	16
11.3	K smallest pair sums: merge sorted rows	17
12	Implementation and proof habits	18
13	Failure modes, exercises, and workflow	19

1 Priority queues as frontier managers

A heap maintains the smallest or largest currently available item. It is the right tool when the next action is determined by one key, but future insertions can change the frontier.

You should be able to say what one heap entry means. Examples:

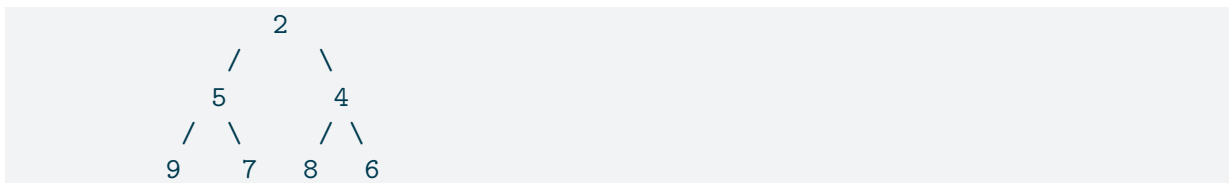
<code>(value)</code>	<code>current candidate number</code>
<code>(end_time)</code>	<code>room that becomes free at this time</code>
<code>(sum, i, j)</code>	<code>pair $A[i]+B[j]$ from stream i at position j</code>
<code>(distance, vertex)</code>	<code>frontier state in Dijkstra</code>

The heap does not prove the algorithm by itself. The proof comes from showing that the smallest or largest key is safe to process next.

2 What a binary heap actually stores

A priority queue is an interface: insert an item and extract the smallest or largest item. A **binary heap** is the usual implementation of that interface. It is a nearly complete binary tree with an ordering rule.

For a min-heap, every parent is at most its children:



This does **not** mean that every level or the underlying array is sorted. It guarantees only that the root is globally smallest. After removing it, the heap repairs the rule and exposes the next smallest item.

2.1 Why heaps are stored in arrays

A complete tree has no gaps, so it can be stored level by level:

tree:	2	5	4	9	7	8	6
0-based i:	0	1	2	3	4	5	6

For a node at index i :

left child	=	$2*i + 1$
right child	=	$2*i + 2$
parent	=	$(i-1)//2$

No pointers are needed. Because the tree is complete, its height is $O(\log n)$.

The diagram shows that the visual tree and the array are two views of the same data. The labels below the cells are array indices.

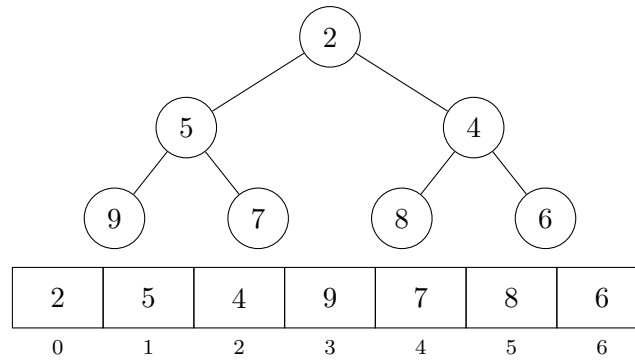


Figure 2.1: The min-heap tree and its level-order array representation.

2.2 Push and pop

To push a value, append it as the last leaf, then swap it upward while it violates the parent rule. To pop the root, move the last leaf to the root, remove the last array entry, and swap the new root downward with its better child.

```

push: append -> move upward at most one tree height
pop:  replace root -> move downward at most one tree height
top:  read the root without changing anything
  
```

Therefore push and pop cost $O(\log n)$, while top costs $O(1)$. Building a heap from all values can be done in $O(n)$, although inserting them one by one gives the simpler $O(n \log n)$ bound.

2.3 Repairing the heap after a push

Suppose 3 is appended to the previous min-heap. It first occupies the next free leaf. Since 3 is smaller than 9, it swaps with 9. It then swaps with 5 and stops below 2.

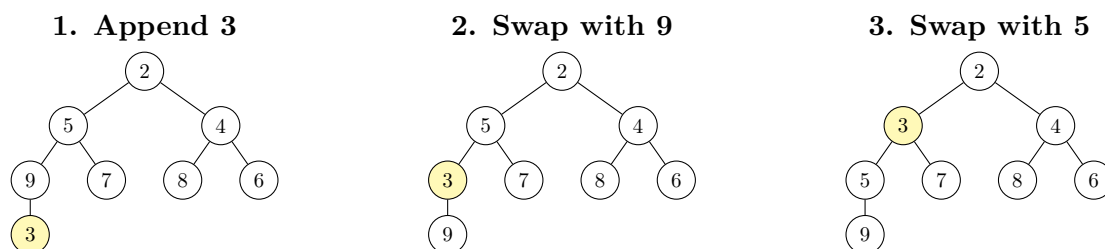


Figure 2.2: Sift-up repairs only the ancestor path of the new leaf.

Only nodes on the new leaf's ancestor path can violate the heap rule. This is why checking and swapping along one path is sufficient.

2.4 Repairing the heap after a pop

After removing the root, move the last leaf into the empty root position. Compare it with both children and swap with the smaller child. Repeat until it is no larger than either child.

Choosing the smaller child matters: swapping with the larger child could leave the new parent larger than its other child. Once again only one root-to-leaf path is touched, which explains the logarithmic cost.

The next diagram removes 2 from the repaired heap. The last leaf, 9, moves to the root. It swaps first with 3, the smaller child, and then with 5.

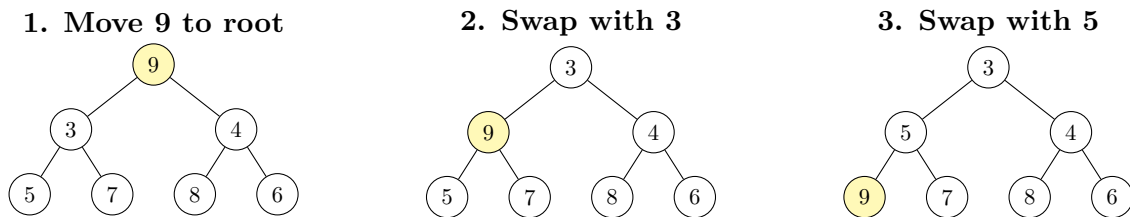


Figure 2.3: Sift-down follows the smaller child until the heap rule holds.

3 Heap versus sorted container

A heap is useful when only the current extreme matters. It cannot efficiently find an arbitrary element, the second-smallest element without removals, or all values in an interval. If the algorithm needs ordered iteration, predecessors, or deletion of arbitrary known values, a balanced ordered container or frequency structure may be more appropriate.

The general recognition question is:

Do I repeatedly need the smallest or largest item among a collection that changes over time?

If yes, a heap may be appropriate. Then state what one heap entry represents and which operations can insert or remove entries.

Some problems maintain a **frontier** of candidates. In that narrower pattern, such as merging sorted streams, ask an additional question: after removing one candidate, can I generate only the candidate that replaces it? This avoids putting every possible future object into the heap at once.

4 Two heaps for medians

Keep the lower half in a max-heap and the upper half in a min-heap:

```
max(low) <= min(high)
len(low) == len(high) or len(low) == len(high)+1
```

Then the lower median is `max(low)`.

Example insertion sequence:

```
insert 5: low=[5], high=[]      median=5
insert 1: low=[1], high=[5]     median=1
insert 9: low=[5,1], high=[9]   median=5
insert 2: low=[2,1], high=[5,9] median=2
```

After every insertion:

1. Put the value in the correct side.
2. Swap tops if order is violated.
3. Rebalance sizes.

A simpler implementation combines the first two steps: compare the new value with the largest value in `low`. Insert it into `low` if it belongs to the smaller half, otherwise into `high`. Then rebalance:

```
if low has two more elements than high:
    move max(low) to high
if high has more elements than low:
    move min(high) to low
```

Why does moving only the top work? If `low` is too large, its maximum is the only element that should cross into the upper half. Every other value in `low` is no larger. The symmetric argument holds for `high`.

Each insertion needs only a constant number of heap operations. Therefore the total cost over n inserted values is $O(n \log n)$, with $O(n)$ stored values.

5 Sweep with ending times

To count rooms for intervals, process intervals by start time. A min-heap stores end times of currently occupied rooms. Before a new interval starts, pop all rooms whose end time is at most the start. The heap size is the number of simultaneous intervals.

For half-open intervals $[l, r)$, an interval ending at time t does not overlap an interval starting at t . Therefore pop while:

```
end_time <= next_start
```

For closed intervals $[l, r]$, the condition would be different.

5.1 Why the maximum heap size is the answer

Immediately before inserting $[start, end)$, remove every ending time at most $start$. What remains is exactly the set of earlier courses overlapping the new course. After pushing end , the heap size is the number of courses active at that moment.

Any schedule needs at least that many rooms because simultaneous courses cannot share. Conversely, expired rooms are always reusable, so the sweep never needs more rooms than this maximum. The lower and upper bounds match.

6 K-way merging

If arrays A and B are sorted, each fixed i gives a sorted stream:

```
A[i]+B[0], A[i]+B[1], A[i]+B[2], ...
```

Put the first item of every stream in a heap. Repeatedly pop the smallest sum and push the next sum from the same stream. This is the same idea as merging many sorted lists.

Worked example:

```
A = [1, 4]
B = [2, 3, 10]
```

Streams:

```
i=0: 3, 4, 11
i=1: 6, 7, 14
```

Each heap entry stores three pieces of information:

```
(pair sum, index in A, index in B)
```

Therefore (3,0,0) represents $A[0] + B[0] = 1 + 2 = 3$, while (6,1,0) represents $A[1] + B[0] = 4 + 2 = 6$.

Initially, put only the first sum of each row in the heap:

```
heap: (3,0,0), (6,1,0)
```

The smallest entry is (3,0,0), so output 3. This removes the current representative of row 0. The next unseen sum in that same row uses the next element of B:

```
A[0] + B[1] = 1 + 3 = 4
```

Push (4,0,1). The heap now contains:

```
(4,0,1), (6,1,0)
```

The next output is consequently 4. After removing it, replace it with the next item of row 0, (11,0,2). The trace begins:

Output	Removed entry	Replacement	Heap afterward
3	(3,0,0)	(4,0,1)	4, 6
4	(4,0,1)	(11,0,2)	6, 11
6	(6,1,0)	(7,1,1)	7, 11

At every moment, the heap stores the first unseen sum from each row, rather than storing all $n \cdot m$ pair sums.

6.1 Why the popped sum is globally next

For each row, the heap contains its first not-yet-reported sum. Every later sum in that row is at least this representative because B is sorted. The smallest heap representative is therefore no larger than any unseen sum in any row. It is safe to output.

After popping row i , only that row loses its representative, so push its next column. This restores the invariant.

The heap has at most n entries. Initialization costs $O(n \log n)$ with individual pushes, and producing k answers costs $O(k \log n)$. The total is $O((n + k) \log n)$ time and $O(n)$ extra memory; it avoids the impossible $O(nm)$ list of all pair sums.

7 Optimal merging: repeatedly combine the two smallest

Suppose merging files of sizes x and y costs $x + y$, and the merged file may be merged again. An early merge contributes its size to later costs, so large files should not be created too early.

The greedy rule is:

```
remove the two smallest current sizes
pay their sum
insert their sum back into the heap
```

For [2, 3, 4, 6]:

merge 2 + 3 = 5	total = 5	heap: 4, 5, 6
merge 4 + 5 = 9	total = 14	heap: 6, 9
merge 6 + 9 = 15	total = 29	heap: 15

Why are the two smallest safe to merge first? In the final merge tree, every original size is charged once for each level above its leaf. Two deepest sibling leaves can be chosen to hold the two smallest weights: exchanging a larger deep weight with a smaller shallow weight cannot increase the total. Contracting those two siblings into one combined weight leaves the same problem on one fewer item. This is the Huffman-coding exchange argument.

There are $n - 1$ merges, each using a constant number of heap operations. The total time is $O(n \log n)$ and the heap uses $O(n)$ memory. Use 64-bit integers: intermediate sizes and the accumulated cost may exceed 32 bits.

8 Python and C++ heap details

Python's `heapq` is a min-heap. For a max-heap, store negative values:

```
import heapq
low = []
heapq.heappush(low, -x)
largest = -low[0]
```

C++ `priority_queue<int>` is a max-heap by default. For a min-heap:

```
priority_queue<int, vector<int>, greater<int>> pq;
```

For pair sums, store all fields needed to generate the next candidate:

```
(current_sum, i, j)
```

If you store only `current_sum`, you cannot know which next pair to push.

9 Checklist

Define exactly what one heap entry represents.
Make sure stale entries are removed or ignored.
For two heaps, rebalance after every insertion.
For interval problems, decide whether touching endpoints overlap.

10 Local exercises

- A. Running Median: After every inserted number, print the lower median of the prefix.
- B. Course Rooms: Given half-open intervals $[l, r)$, print the minimum number of rooms needed.
- C. K Smallest Pair Sums: Two arrays are sorted. Print the k smallest values of $a[i] + b[j]$.
- D. Merge Costs: Repeatedly merge two files while minimizing the total cost.

11 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/39_heaps_priority_queues/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.
Additional worked decision traces

11.1 Running median: preserve two halves

After every insertion, `low` is a max-heap holding the smaller half and `high` is a min-heap holding the larger half. Maintain both facts:

```
every value in low <= every value in high  
size(low) == size(high) or size(low) == size(high)+1
```

For prefix 5, 1, 9, 2, the lower medians are traced below.

insert	low (max heap)	high (min heap)	printed
5	{5}	{}	5
1	{1,5} -> {1}	{5}	1
9	{1}	{5,9} -> move 5	5
2	{1,2}	{5,9}	2

The arrows are the rebalance step. Once the two facts hold, exactly `ceil(prefix_length/2)` values are at most `low.top()`, so it is the requested lower median. Each push/pop costs $O(\log n)$.

11.2 Course rooms: a heap is the active frontier

Sort half-open intervals by start. The min-heap contains end times of courses that have started but not yet ended. For $(0,10)$, $(5,7)$, $(6,20)$, $(10,12)$:

start	interval	remove ends <= start	active end heap after push
0	$[0,10)$	none	{10}
5	$[5,7)$	none	{7,10}
6	$[6,20)$	none	{7,10,20}
10	$[10,12)$	remove 7,10	{12,20}

The maximum heap size is three. The `<=` is essential: half-open intervals ending at time 10 and starting at time 10 do not overlap.

11.3 K smallest pair sums: merge sorted rows

If both arrays are sorted, each fixed i creates a sorted row $a[i]+b[0], a[i]+b[1], \dots$. Seed the heap with each row's first value and advance only the row that was popped. For $a=[1,4,8]$, $b=[2,3,10]$:

```
initial heap: (3,row0,col0), (6,row1,col0), (10,row2,col0)
pop 3, push 4; pop 4, push 11; pop 6, push 7; pop 7, push 14
```

Thus the first four sums are 3,4,6,7. The heap always holds the smallest unreported element of every nonempty row, so its minimum is globally next. Only seed $\min(n,k)$ rows in a generalized variant; the local contract has $k \leq n \cdot m$.

12 Implementation and proof habits

C++. `priority_queue<T>` is a max-heap; write `priority_queue<T, vector<T>, greater<T>>` for a min-heap. Store pair sums in `long long`, and store `(sum,row,col)` as a tuple so lexicographic comparison breaks ties safely.

Python. `heapq` is a min-heap. Negate values for a max-heap, and use tuples such as `(sum,row,col)` rather than custom comparators. Never call `heappop` on an empty heap; prove the seed/advance invariant first.

13 Failure modes, exercises, and workflow

Problem	Wrong shortcut	Test
A	sort every prefix	descending input of length 2e5
B	remove ends < <code>start</code>	[0,10) and [10,12)
C	seed every pair	two arrays of length 2e5 with small <code>k</code>

1. Prove that moving `high.top()` to `low` repairs both running-median facts.
2. Explain why the room heap size after cleanup equals the number of overlapping intervals at the current start time.
3. On paper, list the first five sums for [0,7] and [1,2,9] using rows.

Answers: the moved element is the smallest large-half value; every remaining end is strictly after the current start; the sums are 1,2,8,9,9.

Run `CP_TARGET=solution python3 sections/39_heaps_priority_queues/check.py`. For A compare with a sorted prefix, for B brute-force overlap counts on small interval sets, and for C enumerate and sort every pair sum when `n,m <= 8`.

Use A to maintain the two-heap size invariant. In B, free rooms before allocating a new one. In C, make each heap item represent the next unseen pair. Test duplicate values and equal endpoints.